

OPENREELpy - README

Page 1 - Product Overview

OPENREELpy is a responsive single-page video discovery and streaming application for public-domain, Creative Commons and open-access media. A user enters a VIDEO SEARCH. OPENREELpy converts it into an editable Google advanced-search query restricted to Internet Archive details pages and common web-video formats. At the same time, it requests a structured result index from the Internet Archive public API.

The result index appears in a scrollable modal. Each card identifies the source record, descriptive information, date where available, usage count and actions. Selecting PLAY triggers a second metadata check. OPENREELpy chooses a directly downloadable video file reported by the source record, then opens a new scrollable player modal with native controls and additional play/pause, ten-second seek, speed and fullscreen controls.

This is not a Netflix clone and does not use Netflix branding, content, code, recommendation data or proprietary interfaces. It borrows only familiar catalogue-and-player interaction patterns. No subscription, login, AI key or paid API is required.

Page 2 - Local and Vercel Deployment

Create a Python 3.12 virtual environment, run `pip install -r requirements.txt`, then start locally with `uvicorn app:app --reload --port 8080`. Open <http://localhost:8080>. Docker users can run `docker compose up --build`. The health endpoint is `/api/health`; interactive FastAPI documentation is `/api/docs`.

For Vercel, import the repository or run `vercel deploy` with Vercel CLI 48.1.8 or later. Official FastAPI integration automatically discovers the app instance in the root `app.py`. OPENREELpy intentionally uses this documented zero-configuration route: there is no competing custom entrypoint, legacy builds block, custom Build Command or output directory. `.python-version` pins Python 3.12 and `requirements.txt` supplies production dependencies.

Version 1.1.0 removes the redundant `[tool.vercel]` entrypoint declaration and the plan-dependent function-duration override. This eliminates configuration ambiguity during framework detection. It also makes the static mount tolerant of a cold import while automated tests separately require every interface asset to exist. The health response now exposes application and Python versions, making a successful Vercel invocation directly observable.

`app.py` contains FastAPI routes, safety checks, the advanced-query compiler, Internet Archive integration and playable-file resolver. `templates/index.html` contains the shell. `static` contains CSS and minimal browser JavaScript. FastAPI mounts `/static`; Vercel can promote this supported mount to its CDN. Markdown and PDF manuals remain in `docs` and are served through a strict allowlisted route.

The function returns metadata and direct Archive URLs; it never proxies video bytes. Official references: <https://vercel.com/docs/frameworks/backend/fastapi>, <https://vercel.com/docs/functions/runtimes/python> and <https://vercel.com/docs/functions/limitations>.

Page 3 - Features and Data Flow

The search compiler quotes the cleaned request and restricts the dork to archive.org/details, MP4, WebM or OGV files, plus public-domain, Creative Commons or open-access wording. The query remains editable. COPY transfers it to the clipboard and OPEN GOOGLE opens the user-edited value in a separate tab.

The in-app index uses Internet Archive Advanced Search with mediatype:movies. It retrieves identifiers, titles, descriptions, years, creators and download counts. Thumbnail images use the Archive image service. Selecting a result requests its metadata and filters for non-private MP4, WebM or OGV files. The largest eligible file is selected as the likely primary presentation.

Watchlist and history use localStorage. Recent saved or played items form the Continue Exploring shelf. Saved searches preserve their exact dork. CLEAR removes only OPENREELpy data after confirmation.

Page 4 - Privacy, Security and Accessibility

OPENREELpy has no account database and does not upload viewing history to its own server. Searches contact Internet Archive. Playback contacts the selected Archive download URL. OPEN GOOGLE contacts Google only after deliberate activation. Those services apply their own policies and network logging.

Queries containing explicit piracy, torrent, DRM-bypass, leaked-copy or unauthorised-stream wording are refused. This filter supports responsible use but cannot determine copyright status by itself. Users must inspect the source record and rights statement before reuse or redistribution.

User-controlled text is escaped before HTML rendering. Network values are encoded into URLs. External links use noopener and norereferrer. The player uses native HTML5 video controls and does not execute downloaded files.

The interface supports keyboard navigation, labelled controls, visible focus, high contrast and live native media controls. Dialog content scrolls independently. Desktop grids collapse for tablet and mobile displays.

Page 5 - Limitations and Support

Not every Archive record has a browser-playable file. Some files may be geographically restricted, temporarily unavailable, corrupt, too large or encoded with a codec unsupported by the current browser. OPENREELpy displays a source-record link when playback cannot be verified.

The Google dork is a research aid; OPENREELpy does not scrape Google results. The in-app catalogue is produced independently through the Archive API. Search ranking reflects Archive downloads rather than personal recommendations. There is no adaptive bitrate streaming, DRM playback, offline download manager or cross-device synchronisation.

For questions, use CONTACT. The footer link “Another Website by Julius Olatokunbo” opens <https://www.raiiarcomio.com>. Maintain the rights-aware restrictions, source links, failure messages and tests when extending the project.

If Vercel reports `FUNCTION_INVOCATION_FAILED`, inspect the Function Logs for the first Python traceback, verify Root Directory is the folder containing `app.py`, and redeploy without retaining an old custom Build Command. A successful deployment must return HTTP 200 and `"status":"ok"` from `/api/health`. Do not set the Root Directory to `templates`, `static` or the outer folder that merely contains the project ZIP.